K3s Cluster Build on Proxmox

Phase 2 Runbook — Storage, Secrets, and First Workload

Table of Contents

1. Overview
2. Prerequisites
3. Step 1 — Install Longhorn
4. Step 2 — Install HashiCorp Vault
5. Step 3 — Deploy ChromaDB
6. Step 4 — Cilium Network Policies
7. Step 5 — End-to-End Validation
8. Quick Reference — Placeholder Values

Overview

This runbook picks up from a completed Phase 1 cluster (k3s + Cilium + MetalLB + NGINX Ingress). By the end of Phase 2 you will have distributed persistent storage via Longhorn, enterprise secrets management via HashiCorp Vault, your first production workload (ChromaDB) running on the cluster, and Cilium network policies enforcing least-privilege egress.

Prerequisites

Phase 1 must be complete: 3-node k3s cluster with Cilium, MetalLB, and NGINX Ingress all healthy. Verify before starting:

```
# From your dev machine
kubectl get nodes -o wide
# All 3 nodes should show Ready

kubectl get pods -A
# All pods Running, no restarts
```

Step 1 — Install Longhorn

Longhorn provides replicated block storage across your worker nodes. Each Longhorn volume is replicated to multiple nodes, so you can lose a worker and your data survives. PVCs bind to these replicated volumes.

Verify prerequisites on each worker

`open-iscsi` and `nfs-common` should already be installed from Phase 1. Confirm on each worker:

```
# SSH into each worker and check
ssh <YOUR_USERNAME>@<WORKER1_IP>
systemctl status iscsid.socket
# Should show active (listening)

# iscsid itself will show inactive — that is normal.
# It is socket-activated and starts on demand when
# Longhorn needs an iSCSI connection.

# If not installed:
sudo apt install -y open-iscsi nfs-common
sudo systemctl enable iscsid.socket --now
```

Install Longhorn via Helm

We're adding the Longhorn Helm chart repo and installing it into its own namespace. The replica count is set to 2 so every volume is written to two of our three nodes — enough redundancy for a homelab without burning all our disk space.

```
# From the server node or dev machine with kubectl configured
helm repo add longhorn https://charts.longhorn.io
helm repo update

helm install longhorn longhorn/longhorn \
  --namespace longhorn-system \
  --create-namespace \
  --set defaultSettings.defaultReplicaCount=2 \
  --version 1.10.2
```

Note: `defaultReplicaCount=2` means each volume is replicated across 2 of your 3 nodes. This balances redundancy with disk usage for a homelab. Production environments typically use 3.

Wait for Longhorn to be ready

```
kubectl -n longhorn-system get pods -w
# Wait until all pods show Running (can take 5+ minutes)
```

Note: The `-w` flag streams pod status in real time. You will see `ErrImagePull`, `ImagePullBackOff`, and `CrashLoopBackOff` errors during startup — this is normal. CSI pods race for leader election and restart a few times before settling. If the output appears stalled, press Ctrl+C and check the final state with: `kubectl -n longhorn-system get pods`. If all pods show Running (even with a few restarts), you are good to proceed.

```
# Verify the StorageClass was created
kubectl get storageclass
# You should see 'longhorn' listed
```

Set Longhorn as the default StorageClass

k3s ships with a `local-path` StorageClass that stores data on a single node with no replication. We're swapping the default to Longhorn so any new PVC automatically gets replicated storage.

```
# Remove default from local-path (k3s built-in)
kubectl patch storageclass local-path -p \
  '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'

# Set Longhorn as default
kubectl patch storageclass longhorn -p \
  '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

Validate with a test PVC

Let's prove that Longhorn actually works. We'll create a PersistentVolumeClaim asking for 1GB of storage, spin up a pod that writes a file to it, and then confirm the data is there. A PVC is just

Kubernetes' way of requesting disk space — Longhorn handles the actual replication behind the scenes.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: longhorn-test
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: longhorn
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: longhorn-test-pod
spec:
  containers:
    - name: test
      image: busybox
      command:
        - sh
        - -c
        - echo 'Longhorn works' > /data/test.txt && sleep 3600
      volumeMounts:
        - mountPath: /data
          name: test-vol
  volumes:
    - name: test-vol
      persistentVolumeClaim:
        claimName: longhorn-test
EOF
```

```
# Wait for pod to be ready, then verify
kubectl wait --for=condition=ready pod/longhorn-test-pod \
  --timeout=120s
kubectl exec longhorn-test-pod -- cat /data/test.txt
# Should print: Longhorn works

# Clean up
kubectl delete pod longhorn-test-pod
kubectl delete pvc longhorn-test
```

Step 2 — Install HashiCorp Vault

Vault provides enterprise-grade secrets management with dynamic secrets, automatic rotation, audit logging, and fine-grained access policies. Pods authenticate to Vault using their Kubernetes service accounts and receive secrets via the Vault Agent Injector sidecar.

Install Vault via Helm

We're telling Vault to store its data on our new Longhorn storage, enabling the Agent Injector so it can push secrets into pods as sidecars, and running in production mode — not dev mode — so we learn the real init and unseal workflow.

```
helm repo add hashicorp https://helm.releases.hashicorp.com
helm repo update

helm install vault hashicorp/vault \
  --namespace vault \
  --create-namespace \
  --set server.dataStorage.storageClass=longhorn \
  --set server.dataStorage.size=5Gi \
  --set injector.enabled=true \
  --set server.dev.enabled=false
```

> **Note:** We are running Vault in production mode (`dev.enabled=false`), which requires manual initialization and unsealing. This is the correct approach for learning how Vault actually operates in an enterprise environment.

Wait for Vault pod to start

Vault will come up as 0/1 Running because it's sealed — it can't serve traffic until we initialize and unseal it. That's by design.

```
kubectl -n vault get pods -w
# vault-0 will show 0/1 Running — this is expected
# It needs to be initialized and unsealed before it becomes 1/1
```

Initialize Vault

Initialization creates the encryption keys that protect everything in Vault. We're using Shamir's Secret Sharing — the master key gets split into 5 shares, and you need any 3 of them to unseal. This is how Vault works in production.

```
kubectl -n vault exec vault-0 -- vault operator init \
  -key-shares=5 \
  -key-threshold=3
```

> **Important:** This outputs 5 unseal keys and 1 root token. Save ALL of them in a secure location immediately. You need any 3 of the 5 unseal keys to unseal Vault after every restart. If you lose the keys, you lose access to all secrets.

Unseal Vault

Unsealing is the process of reconstructing the master key from the shares we just created. We feed in 3 of the 5 keys and Vault becomes operational.

```
kubectl -n vault exec vault-0 -- \
  vault operator unseal <UNSEAL_KEY_1>
kubectl -n vault exec vault-0 -- \
  vault operator unseal <UNSEAL_KEY_2>
kubectl -n vault exec vault-0 -- \
  vault operator unseal <UNSEAL_KEY_3>

# Verify — Sealed should now show false
kubectl -n vault exec vault-0 -- vault status
```

Enable the KV secrets engine

Vault uses pluggable secret engines. KV version 2 gives us versioned key-value storage — we can store secrets and Vault keeps a history of changes, which is useful for audit and rollback.

```
# Log in with the root token
kubectl -n vault exec vault-0 -- \
  vault login <ROOT_TOKEN>

# Enable KV v2 secrets engine
kubectl -n vault exec vault-0 -- \
  vault secrets enable -path=secret kv-v2
```

Store your first secret

Now we write the Anthropic API key into Vault. From this point on, the key lives in Vault's encrypted storage — not in environment variables, not in pod specs, not in Git. We're using a demo key here for the walkthrough — replace it with your real key in your own build.

```
kubectl -n vault exec vault-0 -- vault kv put \
  secret/agent/config \
  anthropic-api-key=sk-ant-demo-1234567890abcdef \
  environment=production

# Verify it was stored
kubectl -n vault exec vault-0 -- \
  vault kv get secret/agent/config
```

Configure Kubernetes authentication

This allows pods to authenticate to Vault using their Kubernetes service account tokens. No hardcoded credentials needed.

```
# Enable the Kubernetes auth method
kubectl -n vault exec vault-0 -- \
  vault auth enable kubernetes

# Configure it to talk to the K8s API
kubectl -n vault exec vault-0 -- sh -c '
  vault write auth/kubernetes/config \
    kubernetes_host="https://$KUBERNETES_PORT_443_TCP_ADDR:443"
'
```

> **Note:** This uses the `KUBERNETES_PORT_443_TCP_ADDR` environment variable injected into every pod by Kubernetes. An alternative is `https://kubernetes.default.svc:443` which uses in-cluster DNS. Both are valid.

Create a Vault policy for the agent

Policies control who can read what in Vault. This one says: anything under the agent path is read-only. The agent can fetch its API keys but can never modify or delete them.

```
kubectl -n vault exec vault-0 -- sh -c '
vault policy write agent-policy - <<EOF
path "secret/data/agent/*" {
  capabilities = ["read"]
}
EOF
'
```

> **Note:** This policy grants read-only access to secrets under `secret/agent/`. The agent pod can read its API keys but cannot modify or delete them.

Create the agent namespace and service account

We're creating a dedicated namespace for the agent stack and a service account that Vault will recognize. Think of the service account as the agent's identity card — Vault checks it before handing over any secrets.

```
# Create the namespace where the agent stack will run
kubectl create namespace agent

# Create a service account for the agent
kubectl -n agent create serviceaccount agent-sa
```

Bind the Vault role to the service account

This is the glue between Kubernetes and Vault. We're telling Vault: when a pod shows up with the `agent-sa` service account from the `agent` namespace, trust it and give it the read-only agent policy.

```
kubectl -n vault exec vault-0 -- \
  vault write auth/kubernetes/role/agent-role \
    bound_service_account_names=agent-sa \
    bound_service_account_namespaces=agent \
    policies=agent-policy \
    ttl=1h
```

> **Note:** You may see a warning about audience configuration — this is safe to ignore. The role is created successfully regardless.

Verify the role was created correctly by reading it back:

```
# Verify the role — should show agent-sa, agent namespace, and agent-policy
kubectl -n vault exec vault-0 -- \
  vault read auth/kubernetes/role/agent-role
```

> **Note:** This reads back the role configuration and confirms the binding. You should see `bound_service_account_names` showing `[agent-sa]`, `bound_service_account_namespaces` showing `[agent]`, and `policies` showing `[agent-policy]`. This is your clean confirmation that the role was created correctly.

Test the Vault Agent Injector

This is the payoff. We deploy a test pod with Vault annotations, and the injector automatically adds a sidecar that authenticates to Vault and writes the secret to a file inside the pod. The application just reads a file — it never talks to Vault directly.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: vault-test
  namespace: agent
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "agent-role"
    vault.hashicorp.com/agent-inject-secret-config:
      "secret/data/agent/config"
spec:
  serviceAccountName: agent-sa
  containers:
    - name: test
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
EOF
```

```
# Wait for pod (2/2 — the injector adds a sidecar)
kubectl -n agent wait --for=condition=ready \
  pod/vault-test --timeout=120s

# Verify secret was injected
kubectl -n agent exec vault-test -c test -- \
  cat /vault/secrets/config
# Should show the Anthropic API key

# Clean up
kubectl -n agent delete pod vault-test
```

Step 3 — Deploy ChromaDB

ChromaDB is the vector database that stores the agent's knowledge embeddings. Deploying it as a Deployment with a Longhorn PVC ensures the data persists across pod restarts and node failures.

Deploy ChromaDB

We're deploying three manifests: a PVC that requests 10GB of Longhorn storage for the vector data, a Deployment that runs the ChromaDB container and mounts that storage, and a ClusterIP Service so other pods in the cluster can reach it on port 8000.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: chromadb-data
  namespace: agent
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: longhorn
  resources:
    requests:
      storage: 10Gi
EOF
```

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: chromadb
  namespace: agent
spec:
  replicas: 1
  selector:
    matchLabels:
      app: chromadb
  template:
    metadata:
      labels:
        app: chromadb
    spec:
      containers:
        - name: chromadb
          image: chromadb/chroma:latest
          ports:
            - containerPort: 8000
          volumeMounts:
            - mountPath: /chroma/chroma
              name: data
          resources:
            requests:
              memory: 512Mi
              cpu: 250m
            limits:
              memory: 1Gi
              cpu: 500m
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: chromadb-data
EOF
```

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  name: chromadb
  namespace: agent
spec:
  selector:
    app: chromadb
  ports:
    - port: 8000
      targetPort: 8000
  type: ClusterIP
EOF
```

Verify ChromaDB is running

We'll confirm the pod is healthy, then spin up a temporary curl pod inside the cluster to hit ChromaDB's heartbeat endpoint. If we get a response, the database is up and reachable through Kubernetes service discovery.

```
kubectl -n agent get pods -w
# chromadb pod should show 1/1 Running

# Test the API from inside the cluster
kubectl -n agent run curl-test --rm -it \
  --image=curlimages/curl --restart=Never -- \
  curl -s http://chromadb:8000/api/v2/heartbeat
# Should return a JSON response with a heartbeat timestamp
```

Step 4 — Cilium Network Policies

Lock down the agent namespace so pods can only reach the services they need. This enforces least-privilege egress aligned with OWASP Agentic security principles.

Apply the network policy

This CiliumNetworkPolicy is a whitelist — it defines exactly what the agent pod is allowed to talk to, and everything else gets dropped. DNS for name resolution, the Anthropic API for Claude

calls, SSH ports on our ContainerLab VM for device access, ChromaDB for RAG queries, and Vault for secret retrieval. Nothing else.

```
cat <<EOF | kubectl apply -f -
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: agent-egress
  namespace: agent
spec:
  endpointSelector:
    matchLabels:
      app: agent
  egress:
    # Allow DNS resolution (dns rules required for toFQDNs)
    - toEndpoints:
        - matchLabels:
            k8s:io.kubernetes.pod.namespace: kube-system
            k8s-app: kube-dns
      toPorts:
        - ports:
            - port: "53"
              protocol: ANY
          rules:
            dns:
              - matchPattern: "*"
    # Allow HTTPS to Anthropic API
    - toFQDNs:
        - matchName: api.anthropic.com
      toPorts:
        - ports:
            - port: "443"
              protocol: TCP
    # Allow SSH to ContainerLab VM
    - toCIDR:
        - <CONTAINERLAB_VM_IP>/32
      toPorts:
        - ports:
            - port: "2201"
              protocol: TCP
            - port: "2202"
              protocol: TCP
            - port: "2211"
              protocol: TCP
            - port: "2212"
              protocol: TCP
            - port: "2213"
              protocol: TCP
```

```
            - port: "2214"
              protocol: TCP
    # Allow access to ChromaDB
    - toEndpoints:
        - matchLabels:
            app: chromadb
      toPorts:
        - ports:
            - port: "8000"
              protocol: TCP
    # Allow access to Vault
    - toEndpoints:
        - matchLabels:
            k8s:io.kubernetes.pod.namespace: vault
      toPorts:
        - ports:
            - port: "8200"
              protocol: TCP
EOF
```

> **Note:** Replace `<CONTAINERLAB_VM_IP>` with the IP of the VM running your ContainerLab topology. Adjust SSH ports if your port mappings differ. The DNS rule targets pods labeled `k8s-app: kube-dns`, which matches the default CoreDNS deployment in k3s. If your cluster uses different labels for DNS, adjust the `toEndpoints` match accordingly.

Test the policy

Here's where we prove the policy actually works. We'll deploy a pod with the `agent` label, then try three requests: one to Anthropic that should succeed, one to Google that should timeout

because it's not on the whitelist, and one to ChromaDB that should succeed. If Google fails and the other two work, the policy is doing its job.

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: policy-test
  namespace: agent
  labels:
    app: agent
spec:
  serviceAccountName: agent-sa
  containers:
    - name: test
      image: curlimages/curl
      command: ["sh", "-c", "sleep 3600"]
EOF

kubectl -n agent wait --for=condition=ready \
  pod/policy-test --timeout=60s
```

```
# This should SUCCEED (Anthropic API is allowed)
kubectl -n agent exec policy-test -- \
  curl -s -o /dev/null -w "%{http_code}" \
  https://api.anthropic.com/v1/messages

# This should FAIL / timeout (Google is not allowed)
kubectl -n agent exec policy-test -- \
  curl -s --max-time 5 https://www.google.com

# This should SUCCEED (ChromaDB is allowed)
kubectl -n agent exec policy-test -- \
  curl -s http://chromadb:8000/api/v2/heartbeat

# Clean up
kubectl -n agent delete pod policy-test
```

> **Note:** The Google request timing out while Anthropic and ChromaDB succeed is the proof that the network policy is working. Only explicitly allowed destinations are reachable.

Step 5 — End-to-End Validation

Confirm everything is healthy and working together.

Check all components

One final sweep across the whole cluster. We're checking that every node is healthy, every pod is running, Longhorn has our volumes, Vault is unsealed, ChromaDB is serving, and the network policies are in place.

```
# Cluster nodes
kubectl get nodes -o wide

# All pods across all namespaces
kubectl get pods -A

# Longhorn volumes
kubectl -n longhorn-system get volumes.longhorn.io

# Vault status
kubectl -n vault exec vault-0 -- vault status
# Sealed: false, HA Enabled: false

# ChromaDB health
kubectl -n agent get pods -l app=chromadb

# Network policies
kubectl -n agent get ciliumnetworkpolicies
```

Summary of what you built

Step	Component	What it does
1	Longhorn	Distributed block storage across worker nodes
2	HashiCorp Vault	Secrets management with K8s auth + injection
3	ChromaDB	Vector database — first real workload on the cluster
4	Cilium network policies	Least-privilege egress for the agent namespace
5	Validation	End-to-end test of storage, secrets, and networking

Component	Namespace	Status check
Longhorn storage	longhorn-system	`kubectl get sc` (longhorn = default)
HashiCorp Vault	vault	`vault status` shows Sealed: false
ChromaDB	agent	Heartbeat API returns 200
Network policies	agent	Google blocked, Anthropic allowed
Agent service account	agent	agent-sa bound to Vault role

Phase 2 complete. You now have persistent storage, enterprise secrets management, a running vector database, and locked-down network policies. The cluster is ready to receive the agent application.

Next: Phase 3 — Deploy the FastAPI agent backend, wire up Vault secret injection, connect to ContainerLab devices, and run the first autonomous troubleshooting session.

Quick Reference — Placeholder Values

Fill in your values before starting. These carry forward from Phase 1.

Placeholder	Description	Your Value
`<YOUR_USERNAME>`	SSH username for cluster VMs	
`<SERVER_IP>`	k3s server node IP	
`<WORKER1_IP>`	k3s worker-01 IP	
`<WORKER2_IP>`	k3s worker-02 IP	
`<ROOT_TOKEN>`	Vault root token (from init)	
`<UNSEAL_KEY_1-5>`	Vault unseal keys (save all 5)	
`<YOUR_ANTHROPIC_API_KEY>`	Anthropic API key for Claude	
`<CONTAINERLAB_VM_IP>`	IP of VM running ContainerLab	

Important: Vault unseal keys and the root token are the most critical secrets in this entire build. Store them in a password manager or a physically secure location. If you lose the unseal keys, all secrets stored in Vault become permanently inaccessible.